DurveshChavan / OR Public

Code Issues Pull requests Actions Projects Security and quality Insights

1 Branch 0 Tags Go to file Go to file Code

DurveshChavan Wire client (React+Vite) to FastAPI, expand engine modules, add API t...

5009fa1 · 19 minutes ago

api	Wire client (React+Vite) to Fa...	19 minutes ago
client	Wire client (React+Vite) to Fa...	19 minutes ago
tests	Wire client (React+Vite) to Fa...	19 minutes ago
.gitignore	Wire client (React+Vite) to Fa...	19 minutes ago
README.md	Wire client (React+Vite) to Fa...	19 minutes ago
pytest.ini	Add M0 data layer: UCI 498 l...	last week

README

SLA-Driven Ticket Allocation, Escalation & Hardware Replacement

An IT service desk decision-support system built for MDM40233 (Operations Research). Eight OR techniques -- LP capacity sizing, the transportation problem, Hungarian assignment, Johnson's sequencing rule, decision/game theory, CPM/PERT, and replacement theory -- are implemented from first principles in `api/engine/` and wired into one causal chain: staffing funds who's available, assignment gives tickets to those agents, sequencing orders each agent's work, escalation reviews at-risk tickets and changes their routing, hardware ageing warps assignment cost, and replacement resets it -- see `api/engine/pipeline.py`'s module docstring for the full story.

Architecture

```
client/ (React + Vite)  -->  api/main.py (FastAPI)  -->  api/engine/ (OR
logic)
```

`api/engine/` has no dependency on FastAPI, Streamlit, or React -- it's pure functions over pandas/numpy DataFrames and dicts. `api/main.py` is route handlers and pydantic schemas only (zero OR logic); every number the client shows traces back to an `api/engine` call. `client/` is a plain Vite + React + Tailwind + React Query + recharts app talking to that API over HTTP -- nothing in it recomputes an OR result client-side.

Project layout

```
api/
  engine/            OR logic, one module per technique, no framework imports
    data_layer.py       M0 -- loads/cleans the UCI-498 log, derives empirical
rates
    capacity.py         M1 -- LP staffing/headcount sizing
    transport.py         M2 -- transportation problem (regional ticket
distribution)
    assignment.py        M3 -- Hungarian assignment (ticket -> agent cost
matrix)
    sequencing.py        M4 -- Johnson's two-machine sequencing rule
    decision.py          M5/M6 -- escalation decision theory, EMV, game-
theoretic routing
    cpm_pert.py           -- CPM/PERT network for major-incident timelines
    replacement.py       M7 -- deteriorating-hardware replacement + M7->M3
feedback
    pipeline.py          wires M0-M7 into one run; CONFIG dict holds every
                          non-derivable business assumption, each commented
with why
  main.py              FastAPI app; every route calls straight into
api.engine.pipeline
  requirements.txt
client/
  src/
    pages/             one page per pipeline stage (StaffingPlanner,
TicketAssignment,
                        HardwareHealth, EscalationAdvisor, WorkQueue,
RegionalLoad,
                        MajorIncident, Overview)
    components/        shared UI (Card, Heatmap, StatTile,
EditableNumberField, ...)
    hooks/              React Query hooks wrapping the API client
    lib/                API client, formatting helpers
  vite.config.js        dev-time proxy of /api/* -> localhost:8000
tests/
  test_m0.py .. test_m7.py   one test file per engine module above
  test_pipeline.py           full M0-M7 chain, asserts the feedback loop
actually feeds back
  test_api.py                FastAPI routes via fastapi.testclient
data/                  gitignored; holds the cached UCI-498 CSV after first
run
```

If you're adding a ninth OR technique or reworking an existing one, the engine module is the only place its math should live -- `pipeline.py` should only ever call into it, `main.py` should only ever call into `pipeline.py`, and the corresponding `tests/test_mN.py` should assert the module's output directly (not through the API).

Prerequisites

Python 3.11+ and Node 18+.

Setup

API / engine

```
pip install -r api/requirements.txt
```

Client

```
cd client
npm install
```

The client calls the API via relative paths (e.g. `/api/health`); in dev, Vite proxies `/api/*` to `http://localhost:8000` (see `client/vite.config.js`), so no configuration is needed to run locally. Set `VITE_API_BASE_URL` (e.g. in `client/.env.local`) only if the API is served from a different origin, such as a separate host in production.

Data

The engine runs on UCI dataset 498 (Incident management process enriched event log). `api/engine/data_layer.py`'s `load_incident_log` looks for `data/incident_event_log.csv` first; if it's not there (first run, or a fresh clone -- `data/` is gitignored), it tries `ucimlrepo`, then falls back to downloading the zip directly from the UCI archive, and caches the result to `data/incident_event_log.csv` so every later run is offline. If a run needs to work with no network access at all, see `make_synthetic` in the same file -- `pipeline.py` falls back to it when no cached CSV and no network are available.

Every number that *is* in the dataset comes from a `derive_*` function in `data_layer.py`. Every number that *isn't* (costs, budgets, SLA deadlines, hardware-ageing alpha, ...) lives in the `CONFIG` dict at the top of `pipeline.py`, each entry commented with why it can't be derived -- that's the first place to look when changing a business assumption, and the CLI run below prints a `source:` line under each stage so you can tell which numbers came from which place.

Run

Engine only, CLI:

```
python -m api.engine.pipeline
```

prints the story in order -- capacity funding, regional distribution, the sequenced work queue, the M6-gated escalation routing, and the three-stage (baseline / aged / replaced) cost table -- with a `source:` line under each stage naming exactly where its numbers came from (real UCI-498 log vs. a documented CONFIG assumption).

Full app, two terminals:

```
# terminal 1
uvicorn api.main:app --reload

# terminal 2
cd client; npm run dev
```

Open `http://localhost:5173`.

Tests

```
pytest -q
```

Each `tests/test_mN.py` targets one engine module in isolation; `test_pipeline.py` runs the full M0-M7 chain and checks the M7->M3 feedback loop actually changes assignment cost; `test_api.py` exercises the FastAPI app in-process via `fastapi.testclient`, so it catches route/schema regressions without needing a running server or the client.

Run the relevant `test_mN.py` (or add one) when changing an engine module, and `test_pipeline.py` when changing how modules feed into each other. If a change adds or renames a route in `api/main.py`, extend `test_api.py` rather than relying on manual browser checks alone.

Manual verification checklist

Run the full app (above) and confirm, in the browser:

- [ ] **Overview**: click "Generate" -- summary cards and data-source notes populate.
- [ ] **Staffing Planner**: edit the budget field and re-solve -- funded headcount per skill changes.
- [ ] **Ticket Assignment**: toggle "Show hardware impact" -- the aged total appears alongside baseline and moved tickets are highlighted.

Hardware Health: click "Age + N months" then "Replace recommended" — the three

Releases

No releases published

Contributors 2



DurveshChavan Durvesh Chavan



claude Claude

Languages



Python 70.9% **JavaScript** 27% **CSS** 1.9% **HTML** 0.2%